COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structure and Algorithm	**SECTION:**	CPE21S2
DATE PERFORMED:	10 / 21 / 26	**DATE SUBMITTED:**	10 / 21 / 26
NAME:	Bandigan, Christian Nieles, Cenndy	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 14 **Algorithm Complexity**

1. PROCEDURE OUTPUT

	Best case	Worst case	Analysis
Bubble Sort	O(n) time complexity which refer to input array is already sorted	O(n^2) if the array is sorted or descending order	This algorithm is a line of data comparing the adjacent elements and swapping. Bubble sort in best case O(n) when an already-sorted array comparison and zero swaps While the Worst case O(n^2) when a reverse sorted array forces the maximum number of comparisons and swaps elements into place.
Insertion Sort	Insertion Sort O(n) because the elements are already sorted, so each element only needs to be compared at once.	The worst case of insertion is O(n^2) because the elements are arranged in reverse order. Requiring may comparisons and shifts to sort it correctly	In the best case the list is already sorted.Each key is compared once with the element before shifting is needed so the algorithm makes only n-1 comparisons and the time is O(n). While in the worst case , the list is in reverse order so each key must be compared with shift past every element in the comparisons and the same number of the shift

			on time is O(n^2) .

Table 14-1. Best- Case and Worst- Case Analysis using Theoretical Tools

Comparison Bubble Sort and Insertion Sort
Random

```cpp
#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int size = 10000;

int main(){
  int arr[size];
  int arr2[size];
  srand(time(0));
  for(int i = 0; i < size; i++){
    arr[i] = rand()%100;
  }
  for(int i = 0; i < size; i++){
    arr2[i] = rand()%100;
  }

  std::thread th1(bubbleSort<int>,arr,size);
  std::thread th2(insertionSort<int>,arr2,size);

  th1.join();
  th2.join();

  return 0;
}
```

Output

```
c:\Users\TIPQC\Documents\dsa\HOA14\sortinganlaysis>cd "c:\Users

Start Insertion Sort
Start Bubble Sort

Insertion Sort Runtime: 83 milliseconds
Bubble Sort Runtime: 142 milliseconds
```

Best-Case(Sorted)

HOA14 > sortinganlaysis > main.cpp > main()

```cpp
#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <ctime>
#include "sortingAlgo.h"

const int size = 10000;

int main(){
  int arr[size];
  int arr2[size];
  srand(time(0));
  for(int i = 1; i < size; i++){
    arr[i] = i;
  }
  for(int i = 0; i < size; i++){
    arr2[i] = i;
  }

  std::thread th1(bubbleSort<int>,arr,size);
  std::thread th2(insertionSort<int>,arr2,size);

  th1.join();
  th2.join();

  return 0;
}
```

Worst-Case(Array in descending order)

HOA14 > sortinganlaysis > main.cpp > main()

```cpp
#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <iterator>
#include "sortingAlgo.h"

const int size = 10000;

int main(){
  int arr[size];
  int arr2[size];
  srand(time(0));
  for(int i = 1; i < size; i++){
    arr[i] = i;
  }
  for(int i = 1; i < size; i++){
    arr2[i] =i;
  }

  std::reverse(std::begin(arr), std::end(arr));
  std::reverse(std::begin(arr2), std::end(arr2));

  std::thread th1(bubbleSort<int>,arr,size);
  std::thread th2(insertionSort<int>,arr2,size);

  th1.join();
  th2.join();

  return 0;
}
```

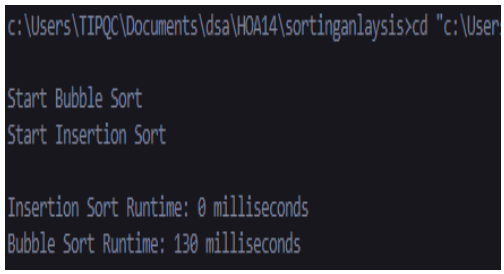
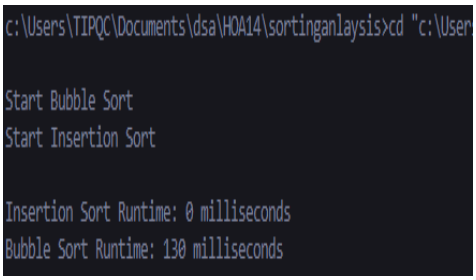
	Bubble Sort	Insertion Sort
Best Case		
Worst Case		

Table 14-2. Best-Case and Worst-Case Analysis using Experimental Tools

Analysis

Based on the results of the experimental analysis, insertion sort is somewhat faster than bubble sort even though their theoretical analysis is the same on both best-case and worst-case.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO A

Theoretical Analysis

Time and Space Complexity

Searching Tecquince	Best Case time	Average case time	Worst case time	
Linear Search	O(1)	O(N)	O(1)	It works in ant any array (sorted or unsorted)

Binary Search	O(1)	O(logN)	O(1)	Most be sorted first

Worst-Case Searching Algorithms (Data is out of bounds)

HOA14 > searchinganalysis > mainsupp.cpp > ExperimentalLinear(int)

```cpp
#include <iostream>
#include <chrono>
#include <thread>
#include <cstdlib>
#include <ctime>
#include <algorithm>
#include <iterator>
#include "searchingAlgo.h"

void ExperimentalLinear(int size);
void ExperimentalBinary(int size);

int main(){

  std::thread th1(ExperimentalLinear,100000);
  std::thread th2(ExperimentalBinary,100000);

  th1.join();
  th2.join();

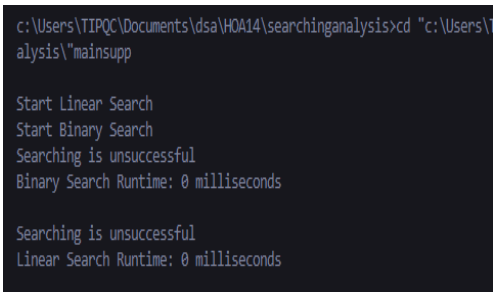
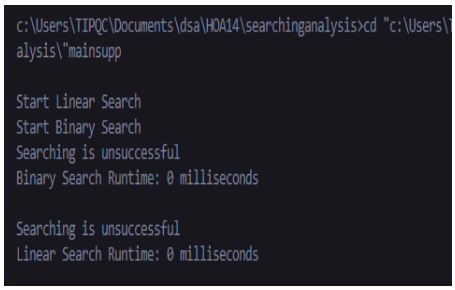
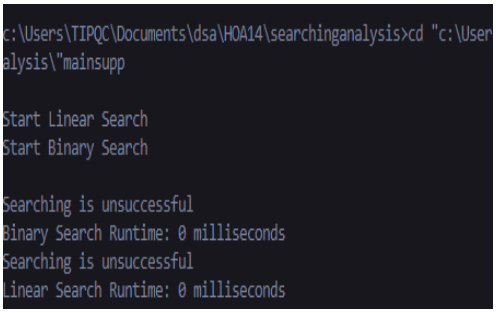
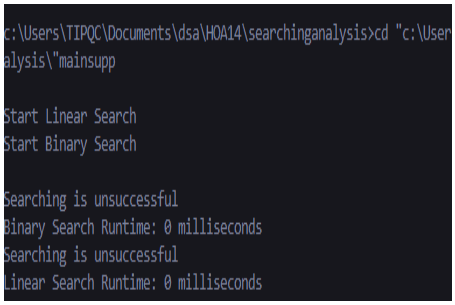
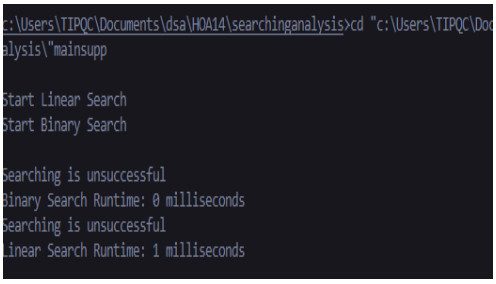
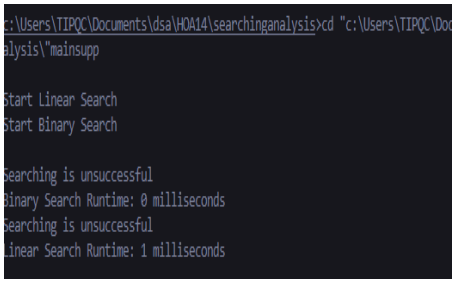
  return 0;
}

void ExperimentalLinear(int size){
  int* arr = new int[size];

  for(int i = 1; i< size;i++){
    arr[i]= i;
  }
  auto start = std::chrono::high_resolution_clock::now();
  std::cout<< "\nStart Linear Search" <<std::endl;
  linearSearchAR(size,100001,arr);
  auto stop = std::chrono::high_resolution_clock::now();
  auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop-start).count();
  std::cout << "Linear Search Runtime: " << duration << " milliseconds\n";
}

void ExperimentalBinary(int size){
  int* arr = new int[size];

  for(int i = 1; i< size;i++){
    arr[i]= i;
  }
  auto start = std::chrono::high_resolution_clock::now();
  std::cout<< "\nStart Binary Search" <<std::endl;
  linearSearchAR(size,100001,arr);
  auto stop = std::chrono::high_resolution_clock::now();
  auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(stop-start).count();
  std::cout << "Binary Search Runtime: " << duration << " milliseconds\n";
}
```

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1, 000		
10, 000		
100, 000		

Analysis

- Based on the results of the experimental analysis, linear and binary search on their worst case doesn't really differ from each other on low inputs like 1000,10000, and 100000.

Conclusion

Bandigan

- In this activity I further improved on multithreading, theoretical and empirical analysis. We were able to compare sorting algorithms specifically bubble sort and insertion sort based on theoretical and empirical analysis. We were able to implement multithreading while conducting empirical analysis on the best and worst case of both sorting algorithms. Moreover, in the supplementary activity, we were able to do the same thing on linear and binary search.

Nieles

- In this activity I learned the complexity of an algorithm measures how much time and space it needs to input the size n grows. And also the best and worst case and the average canvases of every algorithm can

differ a lot and that the worst case is the most common recognition. I compare th are the bubble sort and insertion sort using n = 100, 1,00 and 10, 000 the theoretical they both are O(n) in the best case for sorted list and O(n^2) in the worst case a reversed list

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.